

AdaptiveHop: Adaptive Evidence Retrieval for Multi-Hop Question Answering

Ahmed Yasser Hamad^a, Mohammed Adel Abu Daif^a, Omar Waleed Elsenousy^a

^a*Faculty of Engineering, Alexandria University, Egypt*

Authors and identification numbers

Ahmed Yasser Hamad	9074	Mohammed Adel Abu Daif	9158	Omar Waleed Elsenousy	8873
--------------------	------	------------------------	------	-----------------------	------

Abstract: Retrieval-augmented question answering conventionally retrieves evidence once, scoring passages against the original question. For multi-hop questions we argue that this design fails structurally rather than merely imperfectly: the passage supporting a later reasoning step frequently describes an entity whose name appears nowhere in the question, and becomes lexically reachable only after an intermediate answer is known. No amount of ranking improvement recovers a passage that shares no terms with the query. To address this gap we present *AdaptiveHop*, a retrieval pipeline that decomposes a question into ordered sub-questions, retrieves once per hop with previous-hop answers substituted into the query, accumulates evidence across hops, and terminates retrieval when the accumulated evidence is judged sufficient. The three mechanisms are deliberately built and measured in isolation before integration, so that any improvement remains attributable to a named cause. On the MuSiQue-Ans benchmark, a carefully constructed one-shot baseline establishes that retrieval, not the reader, is the binding constraint: with BM25 at ten retrieval slots, every required supporting paragraph reaches the reader for only 38.0% of questions. We further introduce an evidence-grounding diagnostic which reveals that 59% of *correct* baseline answers are unsupported by the retrieved evidence, indicating that exact match on this benchmark substantially measures parametric recall rather than reading. Under a strictly budget-matched comparison, oracle hop-wise retrieval raises complete-evidence coverage from 30.0% to 81.4% at three slots per hop while consuming fewer unique paragraphs than the condition it beats, and reduces ungrounded correct answers from 59% to 19%. The fully generated end-to-end pipeline, evaluated on 300 development questions, attains exact match 0.153 and answer F1 0.203 using 4.82 paragraphs per question: it exceeds the budget-matched five-slot one-shot baseline on every metric (0.127 EM, 0.169 F1) and more than doubles its complete-evidence coverage (38.0% against 17.3%), while matching the coverage of the ten-slot baseline at less than half the evidence budget. Notably, these gains are attributable to hop-wise retrieval rather than to adaptive stopping, which fired on only 7.3% of questions under the deterministic rule actually deployed. In conclusion, AdaptiveHop demonstrates that conditioning retrieval on intermediate answers converts evidence budget into coverage far more efficiently than enlarging a single retrieval pass, and isolates generated decomposition quality (not retrieval, reading or stopping) as the dominant remaining bottleneck.

Keywords: multi-hop question answering, retrieval-augmented generation, question decomposition, iterative retrieval, adaptive stopping, evidence grounding, MuSiQue.

1. Introduction

Retrieval-augmented generation has become the standard architecture for open-domain question answering. A retriever ranks passages against the question, the highest-scoring few are placed in a language model’s context window, and the model reads them to produce an answer (Lewis et al., 2020). The design rests on an implicit assumption: that the evidence a question requires is lexically or semantically reachable *from the question itself*. Where the answer is contained in a single passage, that assumption holds and retrieval reduces to a lookup.

Multi-hop questions violate it. Consider an example from the benchmark used throughout this report: “*Who is the spouse of the Green performer?*” Answering it requires two facts chained in sequence: that the album *Green* was performed by Steve Hillage, and that Steve Hillage’s spouse is Miquette Giraudy. The passage sup-

porting the second step is about a person whose name occurs nowhere in the original question. A retriever scoring passages against the question text therefore cannot reach it, because the very terms that would identify it become available only once the first step has been answered. The failure is structural rather than a matter of ranking quality: no improvement to the scoring function recovers a passage that shares no terms with the query.

However, while the diagnosis is intuitive, its magnitude is rarely quantified. Multi-hop benchmarks are typically reported through answer accuracy alone, which conflates two distinct abilities (retrieving the right evidence and reading it correctly) and, as we show, a third: recalling the answer from model parameters without reading anything at all. Before a remedy can be evaluated, the constraint it is meant to relieve has to be measured.

In this work we ask whether evidence retrieval can be made *adaptive* (decomposed into reasoning steps, condi-

tioned on intermediate answers, and terminated once the accumulated evidence suffices) such that multi-hop evidence becomes reachable at no greater retrieval cost. We argue that such a design has the potential to overcome two inherent limitations of one-shot retrieval. First, a single query is a single point in term space, whereas a multi-hop question requires evidence scattered across regions of that space which the question does not address. Iterating with substituted answers moves the query to each region in turn. Second, enlarging a one-shot retrieval budget buys coverage only at a punishing cost in precision, since the additional slots are filled by passages the original query ranked poorly, whereas a hop-conditioned query spends each slot against a sharper target.

Motivated by these observations, we build and evaluate three mechanisms on the MuSiQue-Ans benchmark (Trivedi et al., 2022): explicit question decomposition into ordered sub-questions, hop-wise retrieval with previous-answer substitution, and a per-hop adaptive stopping rule. The governing methodological commitment throughout is *separability*. If several components change at once and a score improves, the improvement cannot be attributed to any one of them. Each mechanism was therefore implemented and measured independently, against a baseline built with equal care, before any attempt at integration, and the integrated system is compared against that baseline at a matched evidence budget, not at a matched number of retrieval calls.

1.1. Contributions

1. We construct a validated multi-hop question answering baseline on MuSiQue-Ans and establish that retrieval, not the reader, is the binding constraint: complete supporting evidence reaches the model for only 38.0% of questions even with BM25 at ten retrieval slots.
2. We introduce an evidence-grounding diagnostic measuring how often a *correct* answer arrived without its supporting evidence having been retrieved at all. On the weakest retriever this holds for 79% of correct answers, demonstrating that exact match on this benchmark substantially measures parametric recall rather than reading, and we argue this quantity belongs beside accuracy in any retrieval-augmented evaluation.
3. We implement oracle hop-wise retrieval with previous-answer substitution under a strictly budget-matched comparison, and show it raises complete-evidence coverage from 30.0% to 81.4% at three slots per hop while consuming fewer unique paragraphs than the condition it beats.
4. We provide a mechanism-level explanation of that gain, showing through term-frequency analysis that it arises from substituting a rare, discriminative entity name into later queries rather than from iteration in itself (an account that correctly predicts

where the method is weakest), and we report a stated project hypothesis that the data did not support.

5. We evaluate the fully generated end-to-end pipeline on 300 development questions, where it attains exact match 0.153 and answer F1 0.203 at 4.82 paragraphs per question, exceeding the budget-matched one-shot baseline on every metric and matching the complete-evidence coverage of a baseline using more than twice the evidence. We attribute the gain to hop-wise retrieval rather than to stopping, which fired on 7.3% of questions, and isolate generated decomposition quality as the dominant remaining bottleneck.
6. We contribute a live-trace stopping evaluator that classifies stopping quality against the evidence a trace actually accumulated, which gold-hop-count labelling cannot do once the plan is generated rather than given.
7. We document the experimental infrastructure required to make these measurements trustworthy under free-tier provider constraints: response caching, run provenance and a configuration-resolved component registry, together with the failure modes that motivated each.

2. Related Work

2.1. Multi-Hop Question Answering

Multi-hop question answering benchmarks require combining facts from several documents. HotpotQA (Yang et al., 2018) established the paradigm, pairing questions with gold supporting sentences and introducing supporting-fact evaluation alongside answer accuracy. A recurrent criticism is that many of its questions are answerable through shortcuts, without genuine multi-step reasoning.

MuSiQue (Trivedi et al., 2022) was constructed specifically to resist such shortcuts. Rather than collecting multi-hop questions directly, it composes them from single-hop questions whose answers chain, and screens the result so that no single passage suffices. It additionally publishes a gold decomposition for every question, which the present work exploits both as an evaluation target and as an oracle condition. This report uses the answerable variant, MuSiQue-Ans.

2.2. Retrieval-Augmented Generation

Retrieval-augmented generation (Lewis et al., 2020) conditions a generative model on retrieved passages, decoupling parametric knowledge from an updatable corpus. The standard formulation performs a single retrieval, which is the behaviour treated here as the baseline. Classical sparse retrieval remains a strong and highly interpretable component in this setting. The Okapi BM25 weighting used throughout is the standard probabilistic relevance model (Robertson and Zaragoza, 2009).

Table 1. MuSiQue-Ans composition, reproduced from the released files by the project loader.

Split	2-hop	3-hop	4-hop	Total
Train	14,376	4,387	1,175	19,938
Dev	1,252	760	405	2,417

2.3. Iterative Retrieval and Interleaved Reasoning

Several approaches interleave retrieval with reasoning rather than performing it once. IRCOT (Trivedi et al., 2023) alternates between generating a chain-of-thought sentence and retrieving with that sentence as the query, demonstrating that reasoning state can improve retrieval. Self-Ask (Press et al., 2023) has the model pose explicit follow-up questions before answering, exposing intermediate steps in the generation itself.

The present work is inspired by the reasoning-and-retrieval interaction studied by IRCOT, but it is not an implementation of it. IRCOT conditions retrieval on the most recent generated chain-of-thought sentence. AdaptiveHop represents reasoning explicitly as an ordered list of sub-questions and retrieves for each in turn. This distinction is maintained deliberately throughout.

2.4. Adaptive Retrieval Depth

Not every question requires the same amount of retrieval. Adaptive-RAG (Jeong et al., 2024) predicts question complexity before retrieval and routes each query to one of several fixed strategies, trading cost against accuracy.

The stopping mechanism described in Section 4.5 is a per-hop evidence sufficiency check, not an upfront complexity classifier. The decision is made after each hop, against the evidence actually accumulated, and is therefore a different mechanism from Adaptive-RAG’s router. An upfront classifier remains a plausible future comparison.

3. MuSiQue-Ans: Multi-Hop Question Answering Dataset

3.1. Composition

Every MuSiQue-Ans record contains a natural-language question, a gold answer with alias forms, a gold decomposition enumerating each reasoning step with its own sub-question and answer, and a fixed pool of twenty candidate paragraphs of which only two to four are genuinely supporting. Questions span two to four reasoning hops. Table 1 gives the composition reproduced locally from the released files, which served as an integrity check that the correct dataset variant had been obtained.

Because the official test split is not distributed with gold answers, all evaluation in this report is performed on the development split.

3.2. Distractor Construction

The sixteen to eighteen non-supporting paragraphs per question are not random text. Many were drawn from paragraphs that genuinely supported *other* questions, and are therefore topically plausible. For the worked example of Section 6.2, the distractor pool is built around the lexical ambiguity of the word “Green”: it includes *Green Lake, Minnesota*, the *Green Party of the United States*, and *The Odd Life of Timothy Green*. The benchmark therefore tests whether a retriever can find evidence for *this* question, not evidence that merely appears relevant.

3.3. Schema Validation and Dataset Integrity

The data loader was written with executable schema validation rather than trusting the files to parse: unique identifiers, non-empty questions and answers, contiguous paragraph indices, decomposition ordering, and the requirement that each step’s supporting reference points at a paragraph actually flagged as supporting.

One documented invariant, that every record carries exactly twenty paragraphs, is violated by the official release. Sixteen development records and twenty-one training records ship with as few as sixteen paragraphs. The validator, behaving correctly, rejected the real dataset outright and caused every downstream call to raise. The resolution relaxed the cardinality while preserving the checks that carry genuine detection power: indices must still be exactly $0, \dots, n-1$, so missing, duplicate and out-of-range references remain detectable. The lesson generalises, and recurred: validate against the data that was actually released, not against the schema that was documented.

4. Methodology

4.1. Problem Formulation

Let q be a multi-hop question with an associated closed paragraph pool $P_q = \{p_1, \dots, p_{20}\}$, of which a gold subset $G_q \subseteq P_q$ with $|G_q| \in \{2, 3, 4\}$ is supporting. A retriever returns a ranked subset $R_q \subseteq P_q$ with $|R_q| = k$, and a reader produces an answer $\hat{a}$ from q and R_q .

Two quantities are measured throughout. Recall is $|R_q \cap G_q|/|G_q|$. Complete-evidence coverage, written “all gold” below, is the indicator $\mathbf{1}[G_q \subseteq R_q]$ averaged over questions. The latter is the more consequential: because MuSiQue questions are composed so that no single passage suffices, a question whose evidence set is incomplete is in principle unanswerable from the retrieved context, whatever the reader does.

4.2. Evidence Retrieval

4.2.1. Closed per-question retrieval

Retrieval is performed within each question’s own twenty paragraphs. This is a deliberate scope decision: open-domain retrieval over an external corpus would introduce corpus construction and indexing concerns orthogonal to

the research question, and would confound retrieval quality with corpus coverage.

4.2.2. Indexing and ranking

The retriever is Okapi BM25 (Robertson and Zaragoza, 2009) over the concatenation `title + paragraph_text`, tokenised on alphanumeric sequences. Indexing the title is consequential rather than incidental: MuSiQue titles are typically entity names that the question mentions directly. An earlier lexical-overlap retriever scoring the body alone performs substantially worse (Section 6.1.1), and the title field accounts for much of the difference.

4.3. Question Decomposition

4.3.1. Few-shot prompting and structured output

A language model is prompted with three to five demonstrations drawn strictly from the training split to rewrite q as an ordered list of sub-questions, in which a step depending on an earlier answer references it through an explicit `[ANSWER_N]` placeholder. Development examples are never used as demonstrations. Output is emitted in a structured format and parsed into a typed representation rather than scraped from prose.

4.3.2. Structural validation

Before any expensive metric is computed, each parsed decomposition is checked for hop ordering, declared dependencies, and that every placeholder refers to a step declared as a dependency and occurring earlier. Empty or malformed output is rejected. A parse failure is recorded as a failed prediction. It is not repaired, and the gold decomposition is never substituted in its place.

4.4. Hop-wise Retrieval

4.4.1. Previous-answer substitution

For hop i of a decomposition, the sub-question is instantiated by replacing each placeholder with the answer of the referenced hop, and retrieval is issued on the result. In the oracle configuration of Section 6.1.6 the substituted answers are the gold intermediate answers. In the integrated pipeline they are the model’s own predictions from earlier hops.

Across the full development split, placeholder references were verified to be well formed in every record: never present in a first step, never forward- or self-referential, and never out of range, so substitution is unambiguous.

A detail worth recording concerns steps carrying no placeholder. Table 2 shows that 351 later steps, affecting 14.5% of development questions, have none. These are not implicit dependencies requiring inference. They are genuinely independent hops naming their entity outright, such as *Duane Courtney* » *member of sports team*. MuSiQue contains compositional questions whose hops do not chain. Such steps are retrieved verbatim and counted, never skipped.

Table 2. Dependency structure of gold decomposition steps, full development split.

Category	Count
Total decomposition steps	6,404
First steps (no reference possible)	2,417
Later steps with an <code>[ANSWER_N]</code> reference	3,636
Later steps with no reference	351
Questions containing such a step	351 (14.5%)

4.4.2. Evidence accumulation and the trace

Retrieved paragraphs are accumulated across hops into a deduplicated union, with first occurrence retained so that a paragraph keeps its earliest hop and best rank. Per-hop structure is preserved separately in an evidence trace: a sequence of records each carrying the hop index, the issued query and the paragraphs it returned. The trace is what makes the question “was the evidence sufficient after hop i ?” answerable at all, and it is the interface consumed by the stopping mechanism.

4.4.3. Retrieval budget control

A naive comparison between one-shot and hop-wise retrieval is uninformative: three hops at $k = 3$ retrieves far more than one shot at $k = 3$. Both conditions are therefore given the same number of retrieval slots. Hop-wise spends k_{hop} at each of n hops. One-shot receives exactly $n \times k_{\text{hop}}$ in a single call, a budget that varies per question.

Two consequences matter for interpreting the results. First, because hops overlap, deduplication leaves hop-wise holding fewer unique paragraphs than its slot budget permits (8.6 against one-shot’s 10.6 at $k_{\text{hop}} = 4$), so the comparison is conservative in its favour. Second, the sweep is bounded at $k_{\text{hop}} = 4$ deliberately: at five slots per hop a four-hop question would receive all twenty paragraphs of its closed context, making retrieval trivially perfect and the comparison vacuous.

4.5. Adaptive Stopping

4.5.1. LLM sufficiency check

After each hop, a language model is shown the original question, the evidence accumulated so far, and the current hop’s query as context, and asked whether the evidence suffices to answer confidently. The response is emitted as structured JSON carrying a boolean decision and a justification, and is parsed deterministically rather than by prose matching.

4.5.2. Lexical coverage reference rule

A deterministic alternative stops when the proportion of question tokens present in the accumulated evidence exceeds a threshold. It requires no provider call and is fully reproducible, serving as a reference condition against which the LLM-based rule can be compared, and as a zero-cost component for offline testing of the loop.

Table 3. Live-trace stopping outcomes. Because a trace terminates at the stop, no later prefix exists to compare against. An incomplete trace is therefore classified by what ended it, separating a stopping failure from a retrieval failure.

Outcome	Condition
<code>stopped_at</code>	the stop coincides with h^*
<code>stopped_late</code>	execution continued beyond h^*
<code>stopped_early</code>	stopping ended a trace whose evidence was incomplete
<code>never_sufficient</code>	the whole plan ran and evidence never completed

4.6. The Integrated Adaptive Pipeline

The integrated system chains the three mechanisms. For each question a decomposition is generated once. Its hops are then executed in order, each retrieving with the substituted sub-question and producing an intermediate answer through the reader. After each hop the stopping rule is consulted with the accumulated evidence, and when it signals sufficiency the loop terminates before issuing another executor call, so that a stop costs no further retrieval and no further reader call. The final answer is produced from the original question and the evidence present at the actual stopping point.

Execution reads no gold data. The generated decomposition, the retrieved evidence and the model’s own intermediate answers are the only inputs. Gold decompositions, gold hop answers and supporting-paragraph labels are reserved exclusively for scoring.

4.7. Live-Trace Stopping Classification

Stopping quality on synthetic traces can be judged against gold hop count, because such traces are constructed with one gold paragraph per hop. For a generated trace this is invalid, for two independent reasons. The model chooses its own number of hops, so “hop 3 of 3” and “hop 3 of 2” are not comparable. More subtly, a single hop can retrieve several gold paragraphs at once, as hop 2 of the worked example in Section 6.2 does, so a trace may be complete at hop 1 of a three-hop question.

Classification is therefore performed against what the trace actually accumulated. Let h^* be the smallest hop whose accumulated evidence contains every gold paragraph. With the effective stop taken as the hop at which execution ended, outcomes are assigned as in Table 3.

5. Experiments

5.1. Data Setup

Unless stated otherwise, experiments use a fixed sample of 300 development questions drawn with seed 13, composed of 157 two-hop, 99 three-hop and 44 four-hop records. The sample is held constant across all experiments so that results remain directly comparable.

The sampling procedure requires comment. The development split is ordered by hop count. Taking the first 300 records, the originally specified approach, yields 300

two-hop questions and none of the others, making the required complexity breakdown impossible. A seeded random draw is used instead, which is both representative and exactly reproducible. Retrieval-only experiments, which incur no provider cost, are additionally run over the full 2,417-question split.

5.2. Models and Providers

Table 4 lists the components used. The reader for all reported answer-quality results is `Qwen2.5-7B-Instruct` in 4-bit quantisation, which also generates the decompositions in the integrated run, so that the pipeline and the baselines it is compared against share a single reader. The provider layer is a configuration choice rather than a code path: the pipeline calls a single provider-agnostic entry point, and the backend is selected by configuration. Stopping in the reported integrated run is the deterministic lexical rule and uses no model at all.

5.3. Prompting and Answer Normalisation

A single reader prompt is held fixed across every run being compared. A prompt change is not treated as a legitimate improvement between settings. Each run records a digest of the prompt template so that a silent edit cannot invalidate a comparison unnoticed.

Answers are normalised following the standard SQuAD convention (Rajpurkar et al., 2016) used by the MuSiQue evaluation: lowercasing, punctuation removal, article removal and whitespace collapsing. Exact match is computed against the gold answer and each alias. Token-level F1 takes the best score across alias candidates.

5.4. Hardware and Infrastructure

Retrieval experiments run on CPU and require no provider access. The full development split completes in approximately 100 seconds. Reader experiments run on a free Colab T4 GPU, whose measured throughput is 1.66 seconds per generation, or 300 questions in 8.3 minutes.

Three pieces of infrastructure were necessary to make the measurements affordable and trustworthy under free-tier constraints, and are described with their motivating failures in Section 6.2.3: a response cache keyed on provider, model, prompt and sampling parameters, fast failure on non-recoverable daily quota limits, and per-run provenance recording the provider, model, retriever, seed, sample size, completion status and prompt digest.

5.5. Evaluation Protocols and Metrics

Four families of metric are reported. *Retrieval*: recall, complete-evidence coverage and mean reciprocal rank of the first gold paragraph. *Answer quality*: exact match and token-level F1, overall and by reasoning depth. *Evidence quality*: supporting-evidence precision, recall and F1, computed as a set comparison between predicted and gold paragraph indices following the official MuSiQue supporting-evidence contract. *Stopping quality*: early/correct/late classification on synthetic traces,

Table 4. Components and their roles. Results obtained with different readers are never merged.

Role	Model	Execution
Retriever	BM25 (rank-bm25)	local, deterministic
Reader (reported)	Qwen2.5-7B-Instruct , 4-bit	free Colab T4 GPU
Decomposition (reported)	Qwen2.5-7B-Instruct , 4-bit	free Colab T4 GPU
Stopping (reported)	lexical coverage, threshold 1.0	local, deterministic
Reader (earlier runs)	gpt-oss-120b	hosted API
Stopping (implemented, not run at scale)	qwen3.8-27b	hosted API

Table 5. One-shot retrieval quality, 300-question seeded development sample. “All gold” is the share of questions for which every supporting paragraph was retrieved.

Retriever	Recall	All gold	MRR
$k = 3$			
Lexical overlap	21.1%	2.3%	0.342
BM25	41.2%	8.7%	0.656
$k = 5$			
Lexical overlap	30.9%	6.3%	0.375
BM25	53.4%	17.3%	0.681
$k = 10$			
Lexical overlap	50.9%	18.7%	0.407
BM25	69.1%	38.0%	0.689

and the live-prefix outcomes of Section 4.7 on generated traces.

In addition we report an evidence-grounding diagnostic: among correct answers, the proportion for which the gold supporting set was not fully retrieved. This is not a standard benchmark metric, and is introduced here because it separates reading from recall in a way exact match cannot.

The automated test suite comprises 207 tests, all offline, exercising the loader, both retrievers, the decomposition parser, the scoring implementations, the stopping rule, the adaptive loop and the pipeline.

6. Results and Discussion

6.1. Quantitative Results

6.1.1. Baseline retrieval quality

Table 5 compares BM25 against the lexical-overlap retriever on the seeded sample. BM25 approximately doubles recall at every budget. Mean reciprocal rank nearly doubles as well, indicating that gold paragraphs rank higher rather than merely appearing more often. The decisive figure is complete-evidence coverage: even with the stronger retriever at ten slots, only 38.0% of questions have all of their required evidence present.

6.1.2. Baseline answer quality

Improving retrieval alone more than doubles exact match at $k = 3$ and $k = 5$ (Table 6). By reasoning depth at $k = 10$, BM25 attains EM 0.217 on two-hop questions but only 0.091 on both three- and four-hop questions (less than half), which is the quantitative signature of the structural failure described in the introduction.

6.1.3. Evidence grounding

Absolute accuracy on this benchmark is low and, taken alone, uninformative. A more searching question is whether the system was right for the right reason. The reader is instructed to answer using only the supplied evidence. Compliance is measurable by checking, for each correct answer, whether the gold supporting set was in fact retrieved.

With weak retrieval the majority of “correct” answers were not supported by the retrieved evidence at all, so exact match was substantially measuring parametric recall. Stronger retrieval reduces this from 59% to 19% at $k = 10$: the system does not merely score higher, it begins to score higher for the right reason.

This also resolves an otherwise paradoxical observation. The lexical retriever’s exact match at $k = 3$ (4.7%) exceeds its complete-evidence coverage (2.3%), which is possible only if roughly half of its correct answers required no complete evidence at all.

6.1.4. Supporting-evidence precision, recall and F1

An independent evaluator implementing the official MuSiQue supporting-evidence contract was applied to the same outputs (Table 8). The trend is instructive: increasing k buys recall and loses precision, and for BM25 the F1 peaks at the smallest budget. Retrieving more is therefore not a strategy for improving evidence quality (it dilutes it), which is precisely the argument for retrieving more selectively instead. The lexical retriever shows no such peak, its F1 remaining flat and low across budgets.

6.1.5. Decomposition quality

The single failure in Table 9 is informative rather than incidental: the model referenced [ANSWER_2] in step 4 without declaring step 2 as a dependency, and the validator rejected the parse instead of silently repairing it.

Lexical similarity is a weak proxy for decomposition quality, since a sound paraphrase may score poorly against gold wording. An extrinsic answerability evaluation was implemented for this reason, answering each generated sub-question against its gold supporting paragraph, but it has so far been exercised only as a two-record smoke test (five per-hop checks, token F1 0.087). It is reported as an outstanding gap, not as a measurement.

6.1.6. Hop-wise versus one-shot retrieval

Table 10 reports the central retrieval experiment, under oracle decomposition and matched budgets, over the full

Table 6. One-shot answer quality with the reader held constant (Qwen2.5-7B-Instruct). The retriever is the only variable. All six runs are complete at $n = 300$, seed 13. Depth breakdown shown for $k = 10$.

Retriever	$k = 3$		$k = 5$		$k = 10$		EM by depth at $k = 10$		
	EM	F1	EM	F1	EM	F1	2-hop	3-hop	4-hop
Lexical overlap	0.047	0.065	0.063	0.093	0.113	0.159	0.140	0.081	0.091
BM25	0.107	0.137	0.127	0.169	0.157	0.221	0.217	0.091	0.091

Table 7. Proportion of *correct* answers that arrived without the complete gold evidence having been retrieved.

Retriever	$k = 3$	$k = 5$	$k = 10$
Lexical overlap	79%	68%	59%
BM25	62%	50%	19%

Table 8. Supporting-evidence metrics, 300 records per configuration.

Retriever	k	Precision	Recall	F1
Lexical overlap	3	0.176	0.211	0.188
	5	0.155	0.309	0.204
	10	0.130	0.509	0.204
BM25	3	0.349	0.412	0.371
	5	0.273	0.534	0.355
	10	0.179	0.691	0.281

development split. At $k_{\text{hop}} = 3$ hop-wise retrieval assembles the complete supporting set for 81.4% of questions where one-shot manages 30.0% on an identical budget, while holding fewer unique paragraphs. Since a MuSiQue question is unanswerable without every supporting paragraph, this is the ceiling any downstream reader inherits.

Mean reciprocal rank also rises, from 0.696 to 0.812 at $k_{\text{hop}} = 2$. The advantage peaks at tight budgets and narrows as the budget grows (Δ recall +28.7, +32.3, +27.8, +22.7), which is the expected shape: with enough slots, one-shot eventually brute-forces a twenty-paragraph context. Hop-wise retrieval therefore yields most where budget and latency are constrained. The seeded 300-question sample, evaluated alongside, agrees closely with the full split (86.9% against 87.3% hop-wise recall at $k_{\text{hop}} = 2$), indicating the sample is not an artefact.

6.1.7. Effect of reasoning depth

The project hypothesised that iterative retrieval would help most as reasoning depth increased. The data does

Table 9. Intrinsic decomposition quality, 50 development records, live run.

Metric	Result
Records processed	50
Valid parsed decompositions	49
Validation/generation failures	1
Gold 2/3/4-hop composition	28 / 16 / 6
Overall ROUGE-1	0.630
Overall ROUGE-L	0.574
Overall token F1	0.630
Hop-count accuracy	0.857

not support that hypothesis. At $k_{\text{hop}} = 4$ the recall gain falls monotonically with depth (+27.5, +18.0, +16.4), the opposite of the prediction, and the complete-coverage ordering flips between budgets in a manner consistent with noise.

A different and more defensible claim does hold, and is reported in its place: one-shot’s ability to assemble a complete evidence set collapses with depth, whereas hop-wise degrades gently (Table 11). Hop-wise retrieval does not help more with depth. It prevents the failure that depth otherwise causes.

6.1.8. Adaptive stopping on synthetic traces

Table 12 reports a negative result, stated as such. The deterministic lexical rule requires essentially every question token to appear in the evidence before it will stop, a condition rarely satisfied, so it almost never stops early, and almost never stops on time either. It establishes the measurement machinery and provides a safe, zero-cost reference condition, but it is not a working stopping policy. This prediction was borne out live: deployed in the integrated pipeline of Section 6.1.9, the same rule fired on 7.3% of 300 questions. Whether the LLM-based sufficiency check performs materially better remains untested at scale, and is the one component of the system this report does not evaluate.

6.1.9. Integrated adaptive pipeline

Table 13 reports the fully generated end-to-end pipeline on the complete seeded sample of 300 development questions, alongside the three one-shot baselines of Section 6.1.2 evaluated on the same questions with the same reader. Nothing in this configuration is oracle: the decomposition, the previous-answer substitutions and the final answer are all generated. Parse failures are retained in the denominator and scored as empty predictions, so the reported figures are not conditioned on the pipeline having succeeded.

The comparison that matters is budget-matched, not call-matched. A hop-wise system issuing three retrievals of three slots does not consume nine paragraphs: hops overlap, and duplicates are discarded on accumulation. The pipeline holds 4.82 unique paragraphs per question on average, so the one-shot condition it must be judged against is $k = 5$, not $k = 3$ or $k = 10$. Against that condition the pipeline is better on every metric: exact match rises from 0.127 to 0.153 and answer F1 from 0.169 to 0.203, while complete-evidence coverage more than doubles, from 17.3% to 38.0%.

Table 10. Oracle hop-wise retrieval against one-shot retrieval at matched slot budgets. Full development split, 2,417 questions, BM25.

k_{hop}	Recall			All gold retrieved			Mean unique paragraphs	
	One-shot	Hop-wise	Δ	One-shot	Hop-wise	Δ	One-shot	Hop-wise
1	40.0%	68.7%	+28.7	5.3%	35.4%	+30.0	2.6	2.5
2	54.6%	86.9%	+32.3	17.7%	69.8%	+52.1	5.3	4.6
3	64.4%	92.2%	+27.8	30.0%	81.4%	+51.4	7.9	6.7
4	71.9%	94.6%	+22.7	41.3%	86.9%	+45.6	10.6	8.6

Table 11. Complete-evidence coverage by reasoning depth, full development split.

Condition	2-hop	3-hop	4-hop
$k_{\text{hop}} = 2$			
One-shot	22.4%	14.6%	8.6%
Hop-wise	75.5%	65.4%	60.2%
$k_{\text{hop}} = 4$			
One-shot	41.3%	40.7%	42.7%
Hop-wise	89.4%	82.0%	88.6%

Table 12. Stopping behaviour of the deterministic lexical-coverage rule on synthetic gold traces, 300 records, seed 13.

Subset	n	Early	Correct	Late
Overall	300	0.000	0.003	0.997
2-hop	157	0.000	0.006	0.994
3-hop	99	0.000	0.000	1.000
4-hop	44	0.000	0.000	1.000

Coverage is matched to a baseline using twice the evidence. The more informative comparison is against $k = 10$. The two configurations assemble complete supporting evidence for *exactly* the same number of questions (114 of 300, 38.0% in both cases), but the pipeline does so with 4.82 paragraphs against 10.00. The consequence is visible in precision: evidence precision is 0.270 for the pipeline against 0.179 for $k = 10$, and evidence F1 is 0.367 against 0.281. The pipeline in fact attains the best evidence F1 of any configuration measured in this report except one-shot $k = 3$ (0.371), which reaches that balance only by retrieving so little that it completes the evidence for 8.7% of questions. Answer quality against $k = 10$ is slightly lower (0.153 against 0.157 EM, 0.203 against 0.221 F1), a result we attribute below to parse failures rather than to retrieval.

The gain is concentrated at depth. Table 14 breaks the comparison down by gold hop count. On two-hop questions the pipeline trails $k = 10$ (0.178 against 0.217 EM), which is expected: a two-hop question is the case a large one-shot budget handles best, and the pipeline’s first hop has no previous answer to substitute. On three- and four-hop questions the ordering reverses, and does so despite the lower budget: 0.131 against 0.091 EM at three hops, and 0.114 against 0.091 at four. Where one-shot exact match is flat across depth because deep questions are failed almost uniformly, the pipeline’s advantage grows with the number of hops, which is precisely the regime the method was designed for.

Adaptive stopping contributed almost nothing.

The run deployed the deterministic lexical-coverage rule, not the LLM sufficiency check, and that rule behaved live exactly as Section 6.1.8 predicted it would: it fired on 22 of 300 questions, 7.3%. The mean number of executed hops, 1.95 overall and 2.30 among parsed questions, is therefore set almost entirely by the length of the generated plan rather than by any sufficiency judgement. We state the consequence plainly: the improvements above are attributable to hop-wise retrieval and evidence accumulation, and the adaptive-stopping component of the system remains effectively unevaluated at scale. The efficiency result is a property of decomposition, not of early termination.

Parse failures bound the result, and they are depth-dependent. The decomposition step failed to produce a parseable plan for 46 of 300 questions, 15.3%. Each such question is scored as an empty answer with no retrieved evidence (the strictest available treatment, and the one the project specification mandates). Restricting to the 254 questions where a plan parsed raises exact match to 0.181 and answer F1 to 0.239, both above every one-shot configuration including $k = 10$, and raises complete-evidence coverage to 44.9%. The failures are not uniformly distributed: the parse failure rate rises monotonically with reasoning depth, from 11.5% on two-hop questions to 17.2% at three hops and 25.0% at four. Decomposition therefore fails most often precisely where decomposition matters most, which depresses the headline figures in the depth bands where the method is otherwise strongest.

Plan length explains most of the remaining gap to the oracle ceiling. Oracle hop-wise retrieval on this same sample completes the evidence for 79.7% of questions at three slots per hop (Section 6.1.6). The generated pipeline reaches 38.0%. Table 15 localises the loss. Generated plans average 2.30 hops where the gold chains average 2.58, and the discrepancy is not noise: 29.5% of parsed plans are *shorter* than the gold reasoning chain, and those plans complete the evidence for only 13.3% of their questions. Where the generated plan has the correct number of hops (65.4% of parsed cases), coverage rises to 58.4%. Under-decomposition is thus the single largest identified cause of the shortfall: a plan with too few hops caps the attainable evidence regardless of how well each hop retrieves, and no stopping policy can repair it, because the evidence that was never sought cannot be recovered by declining to stop. The residual gap

Table 13. Integrated adaptive pipeline against one-shot BM25 baselines. All conditions use the same 300 seeded development questions ($n = 300$, seed 13), the same reader and the same prompt. “Paragraphs” is the mean number of unique paragraphs placed in the reader’s context. “Complete” is the share of questions for which every gold supporting paragraph was retrieved. The budget-matched comparison for the pipeline is one-shot $k = 5$.

Configuration	Paragraphs	Answer		Supporting evidence			Complete
		EM	F1	P	R	F1	
One-shot BM25, $k = 3$	3.00	0.107	0.137	0.349	0.412	0.371	8.7%
One-shot BM25, $k = 5$	5.00	0.127	0.169	0.273	0.534	0.355	17.3%
One-shot BM25, $k = 10$	10.00	0.157	0.221	0.179	0.691	0.281	38.0%
Adaptive pipeline, $k_{\text{hop}} = 3$	4.82	0.153	0.203	0.270	0.596	0.367	38.0%
parsed subset ($n = 254$)	5.69	0.181	0.239	–	–	–	44.9%

Table 14. Exact match and answer F1 by gold hop count, adaptive pipeline against the budget-matched and the double-budget one-shot baselines. The pipeline trails at two hops and leads at three and four.

Depth	n	$k = 5$		$k = 10$		Adaptive (EM / F1)
		EM	F1	EM	F1	
2-hop	157	0.166	0.212	0.217	0.283	0.178 / 0.242
3-hop	99	0.091	0.134	0.091	0.156	0.131 / 0.174
4-hop	44	0.068	0.095	0.091	0.149	0.114 / 0.125
All	300	0.127	0.169	0.157	0.221	0.153 / 0.203

Table 15. Pipeline behaviour and cost, 300 questions. Plan-length categories and their coverage are computed over the 254 questions whose decomposition parsed.

Quantity	Value
Questions attempted / configured	300 / 300
Run completed	yes
Parse failures	46 (15.3%)
2-hop / 3-hop / 4-hop	11.5% / 17.2% / 25.0%
Mean unique paragraphs, all	4.82
Mean unique paragraphs, parsed	5.69
Mean executed hops, all	1.95
Mean executed hops, parsed	2.30
Mean gold hop count	2.62
Stopping rule fired	22 / 300 (7.3%)
at hop 1 / 2 / 3	3 / 16 / 3
Plan ran to completion	278 / 300
Plan length vs. gold (parsed)	
exact	166 (65.4%)
too short	75 (29.5%)
too long	13 (5.1%)
Complete evidence by plan length	
exact	58.4%
too short	13.3%
too long	53.8%

between 58.4% and the 79.7% oracle is attributable to sub-question *content* rather than count (wrongly worded or wrongly ordered hops), which the mechanism analysis of Section 6.2.2 characterises.

Table 16. A real trace at $k_{\text{hop}} = 2$, against one-shot retrieval on the same four slots.

Condition	Query	Retrieved
Hop-wise, hop 1	Green » performer	P_1, P_3 (both irrelevant)
Hop-wise, hop 2	Steve Hillage » spouse	P_{10}, P_5 (both gold)
One-shot, $k = 4$	the original question	P_1, P_{14}, P_9, P_{19} (no gold)

Table 17. Document frequency of query terms within the question’s own twenty paragraphs.

Term	In	Consequence
green	16/20	near-useless. Distractors built on this ambiguity
performer	1/20	discriminative, but matches the <i>wrong</i> paragraph
hillage	2/20	rare, and both occurrences are gold

6.2. Qualitative Results

6.2.1. A worked retrieval trace

Aggregate figures establish that hop-wise retrieval works. A single trace explains why. For “*Who is the spouse of the Green performer?*” the gold supporting paragraphs are P_{10} and P_5 . Hop 1 fails outright, missing even its own supporting paragraph. Hop 2 recovers both gold paragraphs, including the one hop 1 failed to find. One-shot retrieval, given the same four slots, finds neither (Table 16).

6.2.2. Why relation-form queries fail

The mechanism is visible in the term statistics. BM25 tokenises on alphanumeric sequences, so **Green » performer** reduces to the two tokens **green** and **performer**. The arrow contributes nothing. Both tokens are defective, in different ways (Table 17).

The gold paragraph for hop 1 describes Steve Hillage as a “musician” and never uses the word “performer”. The relation label is an abstraction that does not occur in the text it points to: a mismatch string matching can-

not bridge. Substituting the intermediate answer replaces this with **hillage**, a rare proper noun occurring in exactly two paragraphs, both of them gold.

Hop-wise retrieval therefore succeeds not because iteration is intrinsically better, but because substituting an intermediate answer replaces a low-signal query with a high-signal entity name, precisely the input a lexical retriever handles well. This also predicts where the method is weakest: the first hop has no previous answer to substitute. Approximately 36% of gold sub-questions use this **subject » relation** form, so the failure mode is systematic rather than anecdotal.

6.2.3. Observed failure modes in experimental infrastructure

Three incidents shaped the project’s experimental practice, and are recorded because each produced a durable safeguard.

Provider substitution. The originally specified commercial API was unavailable. The key on hand belonged to a different provider, and free-tier ceilings later forced a third migration: to a hosted open model, then to a GPU-hosted one. Because the pipeline calls a single provider-agnostic entry point, the reader’s signature never changed and the question-answering code was never edited across three migrations.

A cache that made a wrong configuration look successful. A sweep configured for BM25 produced output byte-identical to an earlier lexical-overlap run: identical checksum, identical retrieved index sets, identical predictions. The retriever function had been passed explicitly while the output filename and provenance were derived from the configuration, so both labelled the result “BM25”. The response cache then turned every call into a hit, so the mislabelled run also completed instantly and cost nothing: it resembled a fast, successful experiment. It was caught by checksum comparison before anything was reported. A cache that makes wrong configurations cheap also makes them plausible. The permanent fixes were structural: a single component registry, a runner that resolves the retriever from configuration and refuses a caller passing a contradicting function, and per-run provenance.

Quota semantics. The hosted provider’s daily token limit is reported only inside the body of a rejected request. No response header exposes it, so rate-based throttling cannot anticipate it. A $k = 10$ sweep over 300 questions requires roughly 256,000 tokens against a 200,000-token daily allowance and is therefore impossible in a single day at any pacing. An earlier attempt at the integrated run was truncated by exactly this limit after four questions. The run reported in Section 6.1.9 was completed only after moving both the reader and the decomposition model onto a local quantised GPU deployment, which removes the daily token ceiling entirely.

6.3. Limitations

The ceiling result is an oracle measurement. Hop-wise retrieval’s 81.4% coverage was measured with gold sub-

questions and gold intermediate answers. It is an upper bound on what a generated decomposition can deliver, not an achievable system result, and the integrated pipeline recovers 38.0% of it. Errors compound in deployment in a way the oracle cannot exhibit: an incorrect first hop poisons every subsequent query.

One component of the system is not evaluated. The integrated run deployed the deterministic lexical stopping rule, which fired on 7.3% of questions. The LLM sufficiency check is implemented and offline-tested but has not been run at scale, so no claim in this report about *adaptive stopping* as a policy should be regarded as established. What is established concerns hop-wise retrieval under a budget constraint.

Decomposition failures are absorbed into the headline figures. Fifteen percent of questions produced no parseable plan and are scored as empty. This is the correct treatment for a system-level number, but it means the reported exact match mixes a retrieval-and-reading result with a plan-generation failure rate, and the two move independently. We report the parsed subset alongside precisely so the two can be separated.

Retrieval quality is not answer quality. Hop-wise retrieval raises the ceiling that exact match and F1 are bounded by. The integrated result shows some of that headroom converting into answers, but the conversion is far from complete: coverage matched to one-shot $k = 10$ yielded slightly lower F1 than $k = 10$, so additional complete evidence is not automatically read correctly.

Evaluation samples are small and unevenly distributed. The main sample is 300 of 2,417 development questions, with only 44 four-hop cases, so per-depth figures carry wide error bars and the depth reversal in Table 14, while consistent in direction across both three- and four-hop bands, rests on few observations in the deepest one. Decomposition was evaluated intrinsically on 50 records.

Results are not comparable across readers. An earlier run with a different model scored EM 0.073 where another scored 0.047 on the same sample. Such results are reported separately and never merged, and every number in Table 13 comes from a single reader on a single sample.

Answers are graded, reasoning is not. A correct answer reached by parametric recall still scores 1, which is why the grounding diagnostic is reported alongside exact match rather than in place of it.

Limited determinism. Despite temperature 0, an identical configuration scored EM 0.063 on one day and 0.073 on another. Differences below roughly one point should be treated as noise. Retrieval-only results are exactly reproducible, and a cached re-run reproduces prior outputs byte-for-byte.

Closed corpus, single retriever. Twenty candidate paragraphs per question and BM25 only. A dense retriever might narrow the lexical gap that hop-wise retrieval exploits, and open-domain retrieval would alter the problem substantially, though it would also enlarge the search space in which a hop-conditioned query has the greater advantage.

6.4. Future Work

Improve decomposition before anything else. The error budget of Section 6.1.9 is unambiguous about priority. Parse failures cost 15.3% of questions outright, and under-decomposition costs most of the remainder: plans shorter than the gold chain complete the evidence for 13.3% of their questions against 58.4% for correctly-sized plans. Constrained decoding or a schema-validated retry would convert much of the parse-failure band directly into evaluable questions, and the depth-dependence of those failures (25% at four hops) means the gain would land where the method is strongest. Neither better retrieval nor better stopping can substitute for this.

Evaluate the LLM stopping rule at scale. The deterministic rule is a safety reference, not a policy: it never stops prematurely and almost never stops on time. With stopping contributing 7.3% of terminations, the adaptive component of the pipeline is currently vestigial, and the natural experiments are to run the LLM sufficiency check over the same 300 questions and to calibrate the lexical threshold below 1.0. Caching the stopping calls, which presently bypass the response cache, is a precondition for affording either.

Strengthen the first hop. The mechanism analysis shows the gain derives from injecting a discriminative entity into later queries. The first hop has no previous answer to inject and is structurally weakest: it failed outright in the worked example, and the pipeline’s only deficit against $k = 10$ is on two-hop questions, where hop 1 carries proportionally the most weight. Query expansion or entity linking applied to hop 1 alone is a targeted and testable intervention.

Address relation-form sub-questions. Where a relation label does not occur in the target text (“performer” versus “musician”), lexical matching cannot succeed. Mapping relation labels to likely surface forms, or routing these cases to a dense retriever, addresses a failure mode now precisely characterised and is the most plausible route from 58.4% coverage on correctly-sized plans toward the 79.7% oracle.

Reduce the cost of a run. Measured from real prompts, the pipeline costs approximately 4,700 tokens per question at $k_{\text{hop}} = 3$. Freezing the already-persisted decompositions removes an entire model stage per iteration and a source of run-to-run variance. Skipping the final-hop sufficiency check is pure waste, since the loop terminates regardless. Together with GPU-hosted readers at the measured 1.66 seconds per generation, these changes make repeated 300-question runs, and therefore ablations, routine rather than exceptional.

Unify the provider layer. Two independent model-consolidation stacks exist, only one of which is cached. Consolidation would remove a maintenance hazard and make stopping calls cacheable immediately.

7. Conclusion

In this work we asked whether evidence retrieval for multi-hop question answering can be made adaptive (decomposed into reasoning steps, conditioned on intermediate answers, and terminated when the accumulated evidence suffices) and whether such a design reaches evidence that a single retrieval pass structurally cannot. We addressed the question by building three mechanisms in isolation, measuring each against a baseline constructed with equal care, and then evaluating the integrated system against that baseline at a matched evidence budget.

Our principal findings are as follows. (1) Retrieval, not the reader, is the binding constraint on this benchmark: a one-shot baseline assembles complete supporting evidence for only 38.0% of questions at ten retrieval slots. (2) A majority of that baseline’s correct answers, 59%, are unsupported by the evidence retrieved, so accuracy here measures reading and parametric recall in a proportion that depends on retrieval quality. We would argue this quantity belongs beside exact match in any retrieval-augmented evaluation. (3) Hop-conditioned retrieval substantially relieves the constraint: under a budget-matched oracle comparison, complete-evidence coverage rises from 30.0% to 81.4% at three slots per hop while holding fewer unique paragraphs than the condition it outperforms, and ungrounded correct answers fall from 59% to 19%. (4) The mechanism is specific and intelligible (substituting an intermediate answer replaces a low-signal query with a rare entity name), and that account correctly predicts the method’s weakest point, the first hop, which has no answer to substitute. (5) The fully generated pipeline converts a substantial part of this into end-to-end performance: at 4.82 paragraphs per question it reaches exact match 0.153 and answer F1 0.203, beating the budget-matched one-shot baseline on every metric and matching the complete-evidence coverage of a baseline that retrieves more than twice as much, with the advantage concentrated on three- and four-hop questions.

Three results run against expectation and are reported unchanged. The project hypothesis that iterative retrieval helps most at greater reasoning depth is not supported in the form stated. What holds instead is that one-shot coverage collapses with depth while hop-wise coverage degrades gently. The deterministic stopping rule stops late on 99.7% of synthetic traces and fired on 7.3% of live questions, so the efficiency we report is a property of decomposition and evidence accumulation, not of early termination: the adaptive-stopping component of the system remains effectively unevaluated. And the binding constraint has moved: it is no longer retrieval but the generated plan. Under-decomposition accounts for most of the shortfall against the oracle ceiling, with plans shorter than the gold chain completing the evidence for 13.3% of their questions against 58.4% for correctly-sized plans, while a further 15.3% of questions yield no parseable plan at all, increasingly so with depth.

In the future, we plan to attack decomposition qual-

ity directly through constrained decoding and schema-validated retries, to evaluate the LLM sufficiency check at the scale the deterministic rule was evaluated at, and to apply query expansion to the first hop, which the mechanism analysis identifies as structurally weakest. More broadly, we believe the evidence-grounding diagnostic introduced here is applicable well beyond this benchmark: wherever a retrieval-augmented system is graded on answers alone, the share of correct answers that arrived without their evidence is a measurable and currently unmeasured confound.

Reproducibility

Every result in this report originates from the project repository:

<https://github.com/GUEST72/Adaptive-Evidence-Retrieval-for-Multi-Hop-Question-Answering>

Retrieval experiments require no provider access and no API key.

Each prediction file is accompanied by a metadata record naming the provider, model, retriever, seed, sample size, completion status and a digest of the prompt template, so no reported result is attributable to an unknown configuration. The six baseline runs underlying Tables 6, 7 and 8 are each complete at $n = 300$.

The integrated result of Table 13 is scored directly from `baseline/results/predictions_adaptive_k3.jsonl`, whose metadata record reports 300 of 300 questions attempted, 46 parse failures, `complete: true`, seed 13, the `bm25` retriever, `Qwen2.5-7B-Instruct` as both reader and decomposition model, and `lexical` stopping. The per-example prediction file and its metadata are the authoritative record. Aggregate report files written during earlier partial runs are superseded by it. Every figure in Tables 13, 14 and 15 was recomputed from that file using the project’s own scoring modules, and the supporting-evidence figures so obtained reproduce the independently stored baseline evidence scores exactly.

The automated test suite comprises 207 passing tests, all offline.

References

- S. Jeong, J. Baek, S. Cho, S. J. Hwang, J. C. Park. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. *NAACL*, 2024.
- P. Lewis, E. Perez, A. Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*, 2020.
- O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, M. Lewis. Measuring and Narrowing the Compositionality Gap in Language Models. *Findings of EMNLP*, 2023.
- P. Rajpurkar, J. Zhang, K. Lopyrev, P. Liang. SQuAD: 100,000+ Questions for Machine Comprehension of Text. *EMNLP*, 2016.
- S. Robertson, H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.
- H. Trivedi, N. Balasubramanian, T. Khot, A. Sabharwal. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the ACL*, 10, 2022.
- H. Trivedi, N. Balasubramanian, T. Khot, A. Sabharwal. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. *ACL*, 2023.
- Z. Yang, P. Qi, S. Zhang, et al. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. *EMNLP*, 2018.